```
#     <rank>1</rank>
#     <url-id>jeremy-kemper</url-id>
#   </contributor>
#   <contributor>
#     <id>4404</id>
#     <name>David Heinemeier Hansson</name>
#     <rank>2</rank>
#     <url-id>david-heinemeier-hansson</url-id>
#   </contributor>
# </contributors>
```

Defined in `active_support/core_ext/array/conversions.rb`.

10.5 Wrapping

The method `Array.wrap` wraps its argument in an array unless it is already an array (or array-like). Specifically:

- If the argument is `nil` an empty list is returned.
- Otherwise, if the argument responds to `to_ary` it is invoked, and if the value of `to_ary` is not `nil`, it is returned.
- Otherwise, an array with the argument as its single element is returned.

```
Array.wrap(nil)       # => []
Array.wrap([1, 2, 3]) # => [1, 2, 3]
Array.wrap(0)         # => [0]
```

This method is similar in purpose to `Kernel#Array`, but there are some differences:

- If the argument responds to `to_ary` the method is invoked. `Kernel#Array` moves on to try `to_a` if the returned value is `nil`, but `Array.wrap` returns `nil` right away.
- If the returned value from `to_ary` is neither `nil` nor an `Array` object, `Kernel#Array` raises an exception, while `Array.wrap` does not, it just returns the value.
- It does not call `to_a` on the argument, though special-cases `nil` to return an empty array.

The last point is particularly worth comparing for some enumerables:

```
Array.wrap(foo: :bar) # => [{:foo=>:bar}]
Array(foo: :bar)      # => [[:foo, :bar]]
```

There's also a related idiom that uses the splat operator:

```
[*object]
```

which in Ruby 1.8 returns `[nil]` for `nil`, and calls to `Array(object)` otherwise. (Please if you know the exact behavior in 1.9 contact fxn.)

Thus, in this case the behavior is different for `nil`, and the differences with `Kernel#Array` explained above apply to the rest of `objects`.

Defined in `active_support/core_ext/array/wrap.rb`.

10.6 Duplicating

The method `Array.deep_dup` duplicates itself and all objects inside recursively with Active Support method `Object#deep_dup`. It works like `Array#map` with sending `deep_dup` method to each object inside.

```
array = [1, [2, 3]]
dup = array.deep_dup
dup[1][2] = 4
array[1][2] == nil  # => true
```

Defined in `active_support/core_ext/object/deep_dup.rb`.